Nova Compiler para CERBERO

Especificación técnica y correspondencia con el proyecto

Proyecto: Compilador de Nova integrado con el procesador CERBERO Curso: CE 1108 — Compiladores e Intérpretes Plataforma destino: CERBERO, ISA VLIW de 128 bits Lenguaje de implementación: C (compilador); Flex y Bison para las fases léxica y sintáctica.

---

1. Propósito y arquitectura del sistema

Nova es un lenguaje imperativo de tipos estáticos. `novac` traduce una fuente `.nv` al ejecutable `.nvx` de la arquitectura CERBERO. El flujo del compilador es:

1. El lexer reconoce lexemas y ubicaciones. 2. El parser valida la gramática y construye un AST. 3. El análisis semántico verifica declaraciones y tipos, gestiona ámbitos y asigna memoria. 4. El generador transforma el AST a código intermedio TAC y lo selecciona, calendariza y traduce a ensamblador VLIW. 5. El ensamblador/resolvedor asigna posiciones finales a etiquetas, relaja saltos largos y resuelve referencias de control. 6. El emisor codifica los bundles y datos en un `.nvx` con encabezado y checksum, compatible con el cargador de imágenes CERBERO.

El compilador es un ejecutable de anfitrión; `.nvx` es código para el procesador CERBERO, no para ejecutar directamente en Linux. Las imágenes `$readmemh` se obtienen con `tools/novac_image.py` o, desde ensamblador, con `novac -x`.

2. Correspondencia con los requisitos del proyecto

Fase 1 — análisis léxico

`fase1_lexico/nova.l` identifica palabras reservadas, identificadores, enteros, cadenas, operadores y delimitadores, y conserva línea y columna. Omite espacios y comentarios. Reporta caracteres inválidos, literales mal formados, cadenas sin cerrar, escapes desconocidos e identificadores demasiado largos. `--tokens` permite inspeccionar la secuencia léxica.

Fase 2 — análisis sintáctico

`fase2_sintactico/nova.y` define la gramática de módulos, declaraciones, funciones, parámetros, tipos, arreglos, inicializadores, bloques, asignaciones, llamadas, expresiones y control de flujo. Bison construye el AST en `fase2_sintactico/ast.c`. Los bloques deben estar balanceados y correctamente anidados; la recuperación de errores permite reportar más de un problema por compilación. `-t` imprime el AST.

Fase 3 — análisis semántico y tabla de símbolos

`fase3_semantico/semantic.c` recolecta declaraciones y luego analiza cuerpos. La tabla distingue variables, parámetros, constantes, funciones y módulos; registra tipos, dimensiones, visibilidad, firmas, ámbito, tamaño y ubicación. Comprueba referencias, argumentos, tipos de retorno, condiciones booleanas, índices, tamaños de arreglos, declaraciones duplicadas, uso de `rumpe` y `perge`, y reglas de intrínsecas.

Las variables globales se asignan en DMEM desde `0x0000`; los literales de cadena comienzan en `0x2000`; el buffer de salida para `scribe`/`dic` empieza en `0x3000`. Los locales y temporales se asignan en marcos de pila, usando R14. `--sym` muestra la tabla y el resumen de memoria. `importa` acepta rutas relativas, alias `ut`, rutas adicionales `-I` y objetos de fuente `.nvo`.

Fase 4 — generación de ensamblador

`fase4_codigo/tac.c` construye código intermedio de tres direcciones, sin acoplar las expresiones del lenguaje a una instrucción concreta. El selector `fase4_codigo/codegen.c` traduce el TAC a los cinco slots de CERBERO, asigna temporales y registros, genera prólogos/epílogos de función y emite instrucciones de aritmética, memoria, lógica, llamadas y control.

El generador respeta la latencia arquitectónica de tres bundles entre productor y consumidor y el bundle de delay slot posterior a los saltos. `-s` muestra el ensamblador y además guarda un `.asm`; `--ir` muestra TAC; `-p` presenta cada bundle con dirección, slot e bytes codificados. `-v` muestra el avance de las fases.

Fase 5 — cálculo de saltos y referencias

El generador conserva las etiquetas hasta que el tamaño de todas las funciones y rutinas de soporte es conocido. Calcula los desplazamientos relativos de saltos y llamadas; los saltos incondicionales fuera del rango del formato corto se expanden a secuencias de salto indirecto. La expansión se repite y vuelve a calcular las posiciones hasta converger. Las direcciones absolutas de las etiquetas se sustituyen después del layout definitivo. El ensamblador independiente CERBERO también valida etiquetas duplicadas, ausentes y desplazamientos fuera de rango.

Fase 6 — generación de código binario

Cada bundle ocupa 16 bytes (128 bits) y tiene los slots en el orden arquitectónico S0, S1, S2, S3, S4. Los formatos y códigos de operación se mantienen alineados con los `define` del RTL CERBERO; el ensamblador independiente lee la misma tabla de `src/cpu/cerbero_defs.svh`.

El ejecutable `.nvx` lleva un encabezado de 48 bytes:

- `0..3`: magia `NOVA`.
- `4..5`: versión 1; `6..7`: reservado.
- `8..11`: cantidad de bundles de código.
- `12..15`: dirección de carga del código en IMEM.
- `16..19`: punto de entrada.
- `20..23`: dirección inicial de datos en DMEM.
- `24..27`: bytes inicializados incluidos en el archivo.
- `28..31`: bytes de datos que el cargador debe poner en cero.
- `32..35`: tope inicial de pila.
- `36..39`: checksum de código y datos inicializados.
- `40..47`: reservado para el enlazador.

Los enteros del encabezado son little-endian. Después del encabezado se almacenan los bundles y, a continuación, los datos inicializados. El conversor valida versión, campos reservados, checksum, alineación y límites de memoria. CERBERO admite 2048 bundles de IMEM y 64 KiB de DMEM; se reporta error si se superan.

3. Resumen del lenguaje Nova

Léxico

- Identificadores: `[A-Za-z_][A-Za-z0-9_]*`, hasta 255 caracteres.
- Enteros: decimales y hexadecimales `0x...`, con semántica de palabra

`integer` de 32 bits con signo.

- Cadenas: delimitadas por `"..."`; escapes admitidos: `\n`, `\t`, `\\` y `\"`.
- Comentario de línea: `# comentario`.
- Comentario de bloque: `#{ comentario multilínea }#`.
- Palabras reservadas: `functio`, `varia`, `constans`, `privatum`, `importa`, `ut`, `redde`, `si`, `sin`, `aliter`, `dum`, `per`, `rumpe`, `perge`, `integer`, `veritas`, `textus`, `nihil`, `verum`, `falsum`, `et`, `aut` y `non`.

Los operadores incluyen asignación y asignaciones compuestas; suma, resta, multiplicación, división y módulo; comparación; operadores bit a bit; desplazamientos; negación; y conjunción/disyunción lógica de corto circuito. La precedencia está definida explícitamente en la gramática Bison.

Tipos, declaraciones y funciones

Los tipos escalares son `integer`, `veritas`, `textus` y `nihil`. Los arreglos admiten una o dos dimensiones; los parámetros pueden especificar dimensiones abiertas, como `integer[]` o `integer[][]`. No hay tipos de punto flotante. `constans` declara valores globales inmutables; `varia` declara variables locales. Las variables globales mutables no se admiten.

```
constans LIMITE : integer = 5;

functio factorial(n : integer) -> integer {
    varia resultado : integer = 1;
    per (varia i : integer = 1; i <= n; i += 1) {
        resultado *= i;
    }
    redde resultado;
}

functio principale() -> integer {
    redde factorial(LIMITE);
}
```

Una función se declara con `functio nombre(parametros) -> tipo { ... }`. `redde` retorna un valor o termina una función `nihil`. Se admiten llamadas, recursión de cola anotada con `@cauda`, visibilidad privada con `privatum` e importaciones con alias. La función ejecutable de entrada se llama `principale`.

Control y arreglos

- `si (condicion) { ... }`, con rama opcional `sin si (...) { ... }` y/o `aliter { ... }`.
- `dum (condicion) { ... }`.
- `per (inicializacion; condicion; asignacion) { ... }`.
- `rumpe` sale del ciclo actual y `perge` continúa la siguiente iteración.
- Inicialización de arreglos con listas entre llaves y acceso mediante uno o dos índices. En dos dimensiones se usa orden por filas.
- `longitudo(arreglo)` obtiene la dimensión correspondiente. Los índices se verifican en ejecución; una violación termina con el código de ejecución documentado (134).

Funciones intrínsecas y criptografía

`scribe` y `dic` escriben en el buffer de salida que observa el simulador. `ingredere` inicia autenticación; `custodi` instala una clave; `feistel` y `desfeistel` ejecutan cifrado y descifrado F4E/F4D en el slot criptográfico. El código que accede a la bóveda se ubica en la región segura de IMEM `0x7000..0x7FFF`. La descripción completa de opcodes, registros, formatos, latencias, pila, memoria, autenticación y fallas está en `../docs/isa.md`.

4. Invocación y formatos de salida

Desde `nova_compiler-alineaci-n/`:

```
make
./novac programa.nv -o programa.nvx
./novac -s programa.nv -o programa.nvx
./novac -p programa.nv -o programa.paquetes.txt
./novac -x programa.asm -o programa.hex
python3 tools/novac_image.py programa.nvx --imem programa.imem.hex --dmem programa.dmem.hex
python3 tools/simular.py programa.nvx
```

`-s` guarda `programa.asm` además de `programa.nvx`, y mantiene el ensamblador en stdout. `-x` produce una imagen IMEM para `$readmemh` y los sidecars `programa.hex.dmem.hex` y `programa.hex.lst`; no sustituye el formato empaquetado `.nvx`. Los errores de uso, compilación y E/S retornan 2, 1 y 3, respectivamente.

5. Ejercicios y resultados esperados

Las pruebas de entrega compilan cada fuente y ejecutan su binario en el testbench `test/tb_nova_rtl.sv` sobre el RTL CERBERO:

- Factorial: calcula `5!`; retorno R6 = 120.
- Búsqueda 2D: recorre una matriz de enteros y devuelve la posición

fila/columna del valor 6 codificada como `fila * 10 + columna`; R6 = 12.

- Mergesort iterativo: ordena ocho elementos; comprueba el orden creciente

y codifica extremos 1 y 82; R6 = 192 si se cumple la condición.

- Matriz con `for`: inicializa y suma; R6 = 138.
- Multiplicación de matrices: calcula producto 2×3 por 3×2; R6 = 415.
- Criptografía: cifrado F4E y descifrado F4D de una ronda, con clave y

vector conocidos; ambos testbenches comprueban el bloque y autenticación.

Las tres entregas principales respetan el tamaño de IMEM: factorial 166 bundles, búsqueda 683 y mergesort 1216. Las pruebas adicionales también comprueban arreglos, ciclos anidados, llamadas, módulos importados, objetos, aritmética, errores de rango/división, autenticación y comparación de la codificación frente al ensamblador CERBERO independiente.

6. Construcción y verificación reproducibles

Requisitos de anfitrión: GCC, Make, Flex, Bison, Python 3 e Icarus Verilog (`iverilog` y `vvp`) para el RTL.

```
make clean
make
make prueba
make prueba-rtl
make depurar F=todo
```

`make prueba` valida lexer, parser, semántica, diagnósticos y las opciones `-s`, `-p` y `-x`. `make prueba-rtl` ejecuta los programas integrados sobre el procesador. `make depurar F=todo` añade recuperación de errores, sanitizers, verificación de tabla de códigos y contraste binario contra el ensamblador independiente. Los informes concretos están en `evidencia/`.

7. Limitaciones declaradas

- El lenguaje no tiene punto flotante, memoria dinámica de usuario, structs ni

clases; las palabras reservadas futuras se rechazan como tales.

- `.nvo` es un contenedor versionado de fuente con checksum, no un objeto

relocatable independiente de la fuente. Las importaciones se vuelven a analizar y se enlazan antes de generar código.

- El buffer de salida está destinado al simulador y a herramientas del curso;

la ISA no define una consola de hardware.

- La pila, DMEM e IMEM son recursos finitos de CERBERO; los límites se validan

o se reportan durante la compilación/carga.